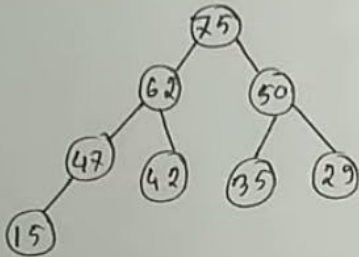


Max Heap Insertion and Deletion

Md. Farhan Hossain
B.Sc. in CSF, UAP
farhanhossain246@gmail.com

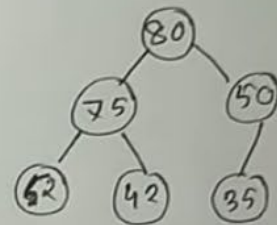
Insertion



1	2	3	4	5	6	7	8	9
75	62	50	47	42	35	29	15	

1	2	3	4	5	6	7	8	9

Deletion

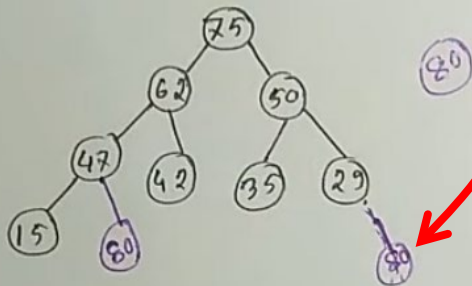


1	2	3	4	5	6
80	75	50	62	42	35

“all levels except the last are completely filled, and in the last level all nodes are to the left as much as possible.”

Since heap is a complete binary tree we cannot insert 80 here

Insertion

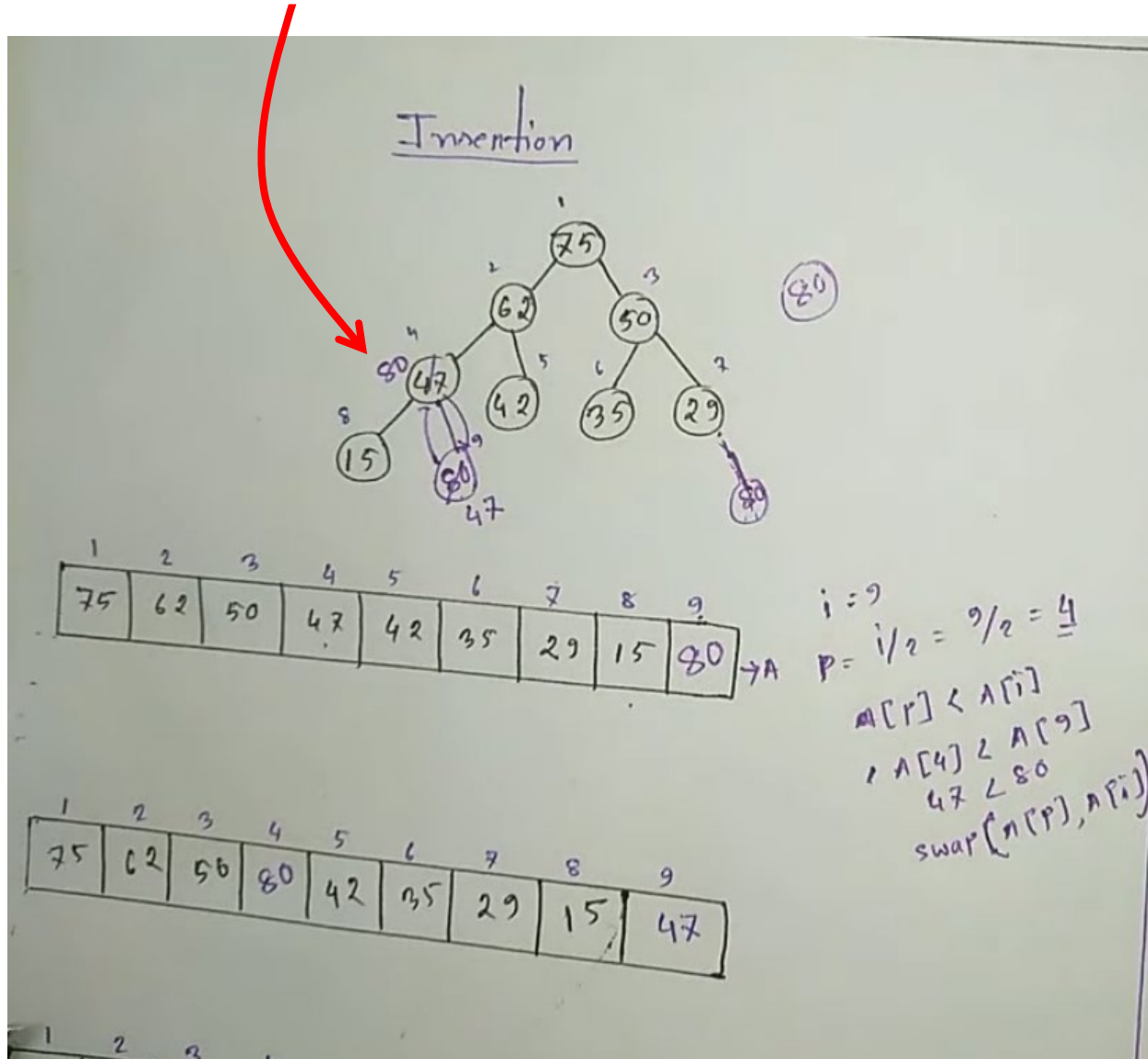


1	2	3	4	5	6	7	8	9
75	62	50	47	42	35	29	15	80

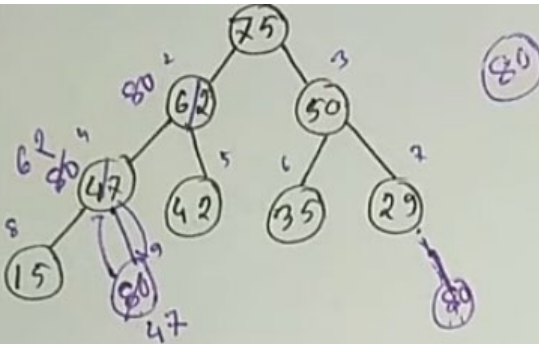
1	2	3	4	5	6	7	8	9

Step 1

Since in maxheap parent cannot be smaller than child, therefore we swap them in each step



Step 2



1	2	3	4	5	6	7	8	9
75	62	50	47	42	35	29	15	80

→ A

$i = 9$
 $P = i/2 = 9/2 = 4$
 $A[P] < A[i]$
 $A[4] < A[9]$
 $47 < 80$
 $\text{swap}(A[P], A[i])$
 $i = P(4)$

1	2	3	4	5	6	7	8	9
75	62	50	80	42	35	29	15	47

↑

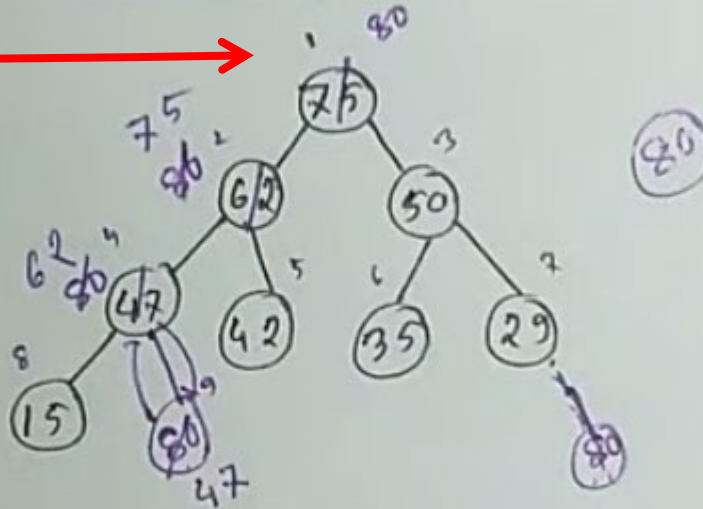
$P = i/2 = 4/2 = 2$
 $62 < 80$
 $\text{swap}(A[P], A[i])$
 $i = P(2)$

1	2	3	4	5	6	7	8	9
75	80	50	62	42	35	29	15	47

↑

Step 3

Insertion



1	2	3	4	5	6	7	8	9
75	62	50	80	42	35	29	15	47

1	2	3	4	5	6	7	8	9
75	80	50	62	42	35	29	15	47

1	2	3	4	5	6	7	8	9
80	75	50	62	42	35	29	15	47

$$i = P(1) = 4$$

$$P = i/2 = 4/2 = 2$$

$62 < 80$
 $\text{swap}(A[P], A[i])$

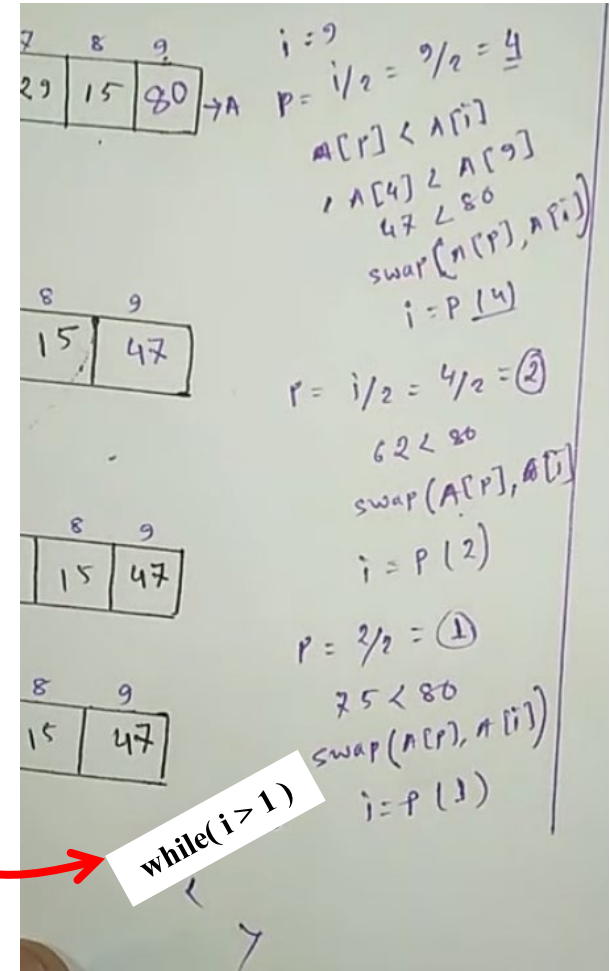
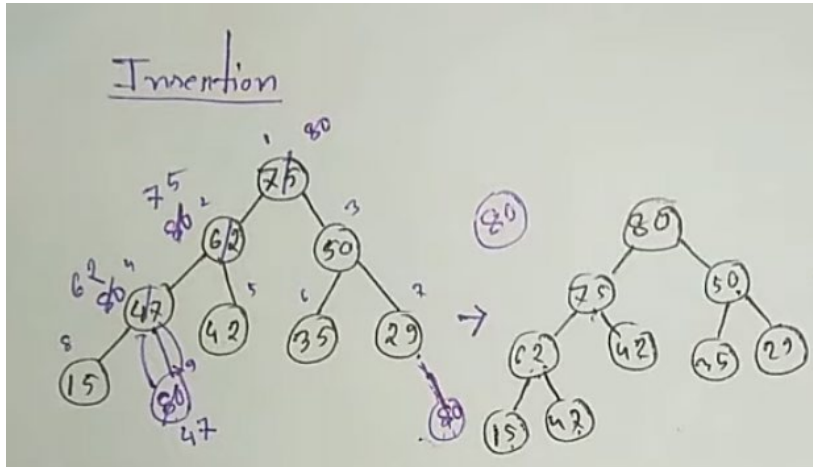
$$i = P(2) = 1$$

$$P = 2/2 = 1$$

$75 < 80$
 $\text{swap}(A[P], A[i])$

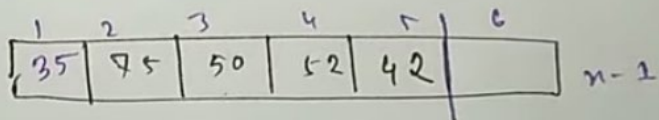
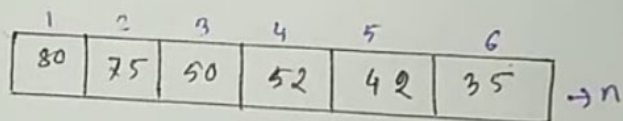
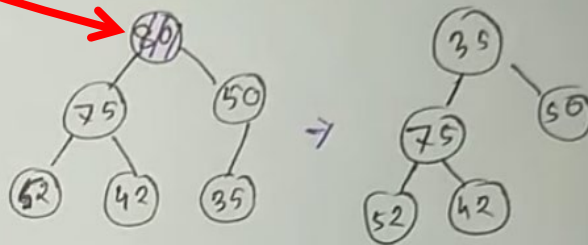
Step 4

Loop will run for



Deletion starts from the root

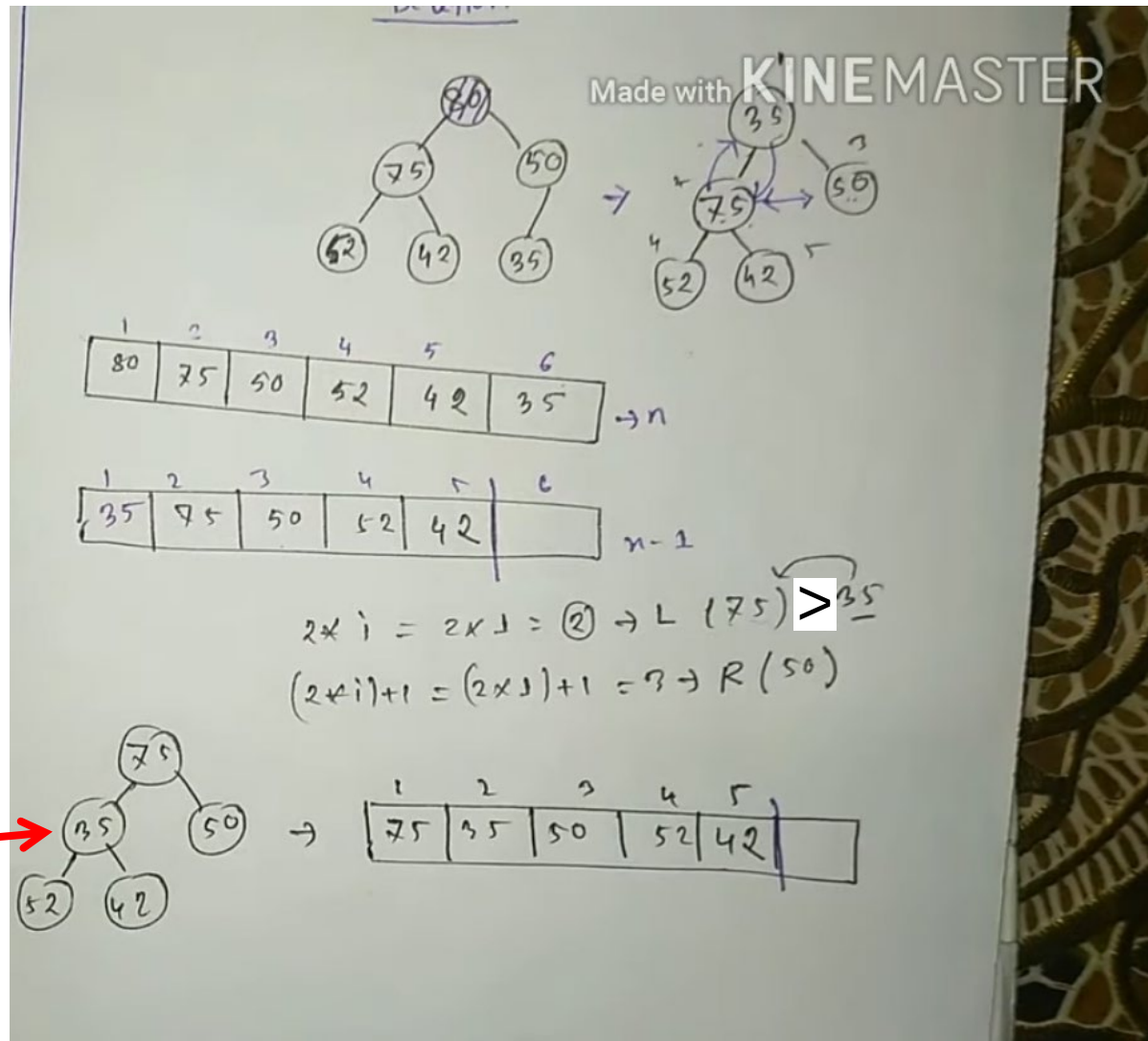
Deletion



Since in maxheap parent cannot be smaller than child, therefore we swap "the greater child" with smaller root in each step

Step 1

Greater of 75 and 50 was swapped with smaller root 35



Step 2

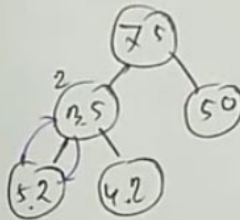
1	2	3	4	5	6
80	75	50	52	42	35

Made with KINEMASTER

1	2	3	4	5	6
35	75	50	52	42	

$$2 \times i = 2 \times 1 = 2 \rightarrow L(75) \quad \cancel{35}$$

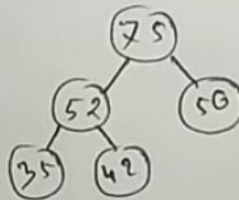
$$(2 \times i) + 1 = (2 \times 1) + 1 = 3 \rightarrow R(50)$$



1	2	3	4	5	
75	35	50	52	42	

$$2 \times 2 = 4 \rightarrow L(52) \quad \cancel{35}$$

$$2 \times 2 + 1 = 5 \rightarrow R(42)$$

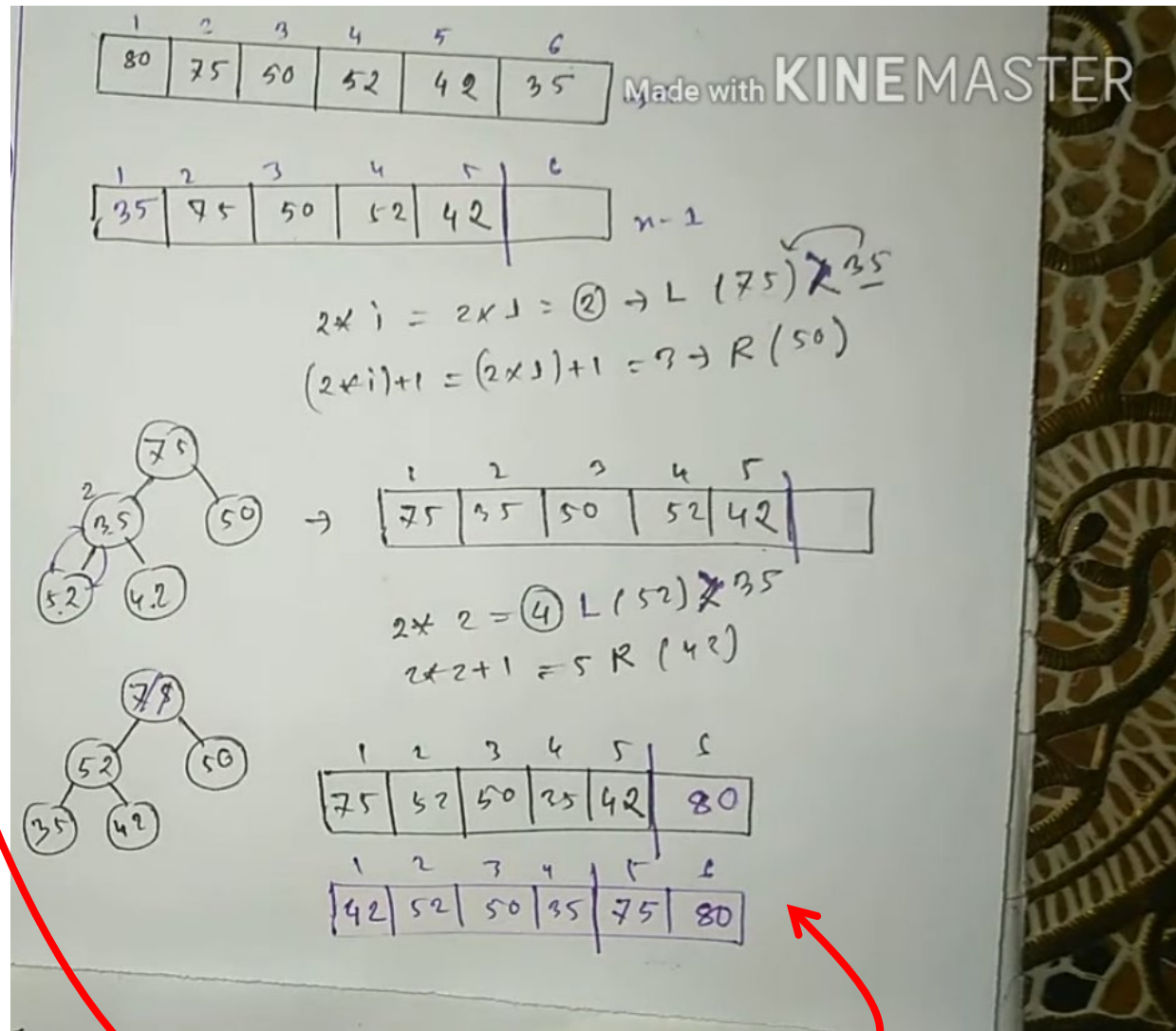


1	2	3	4	5	6
75	52	50	35	42	

Step 3

Deleted values are stored in the unused index

Deletion sorts the deleted values in ascending order



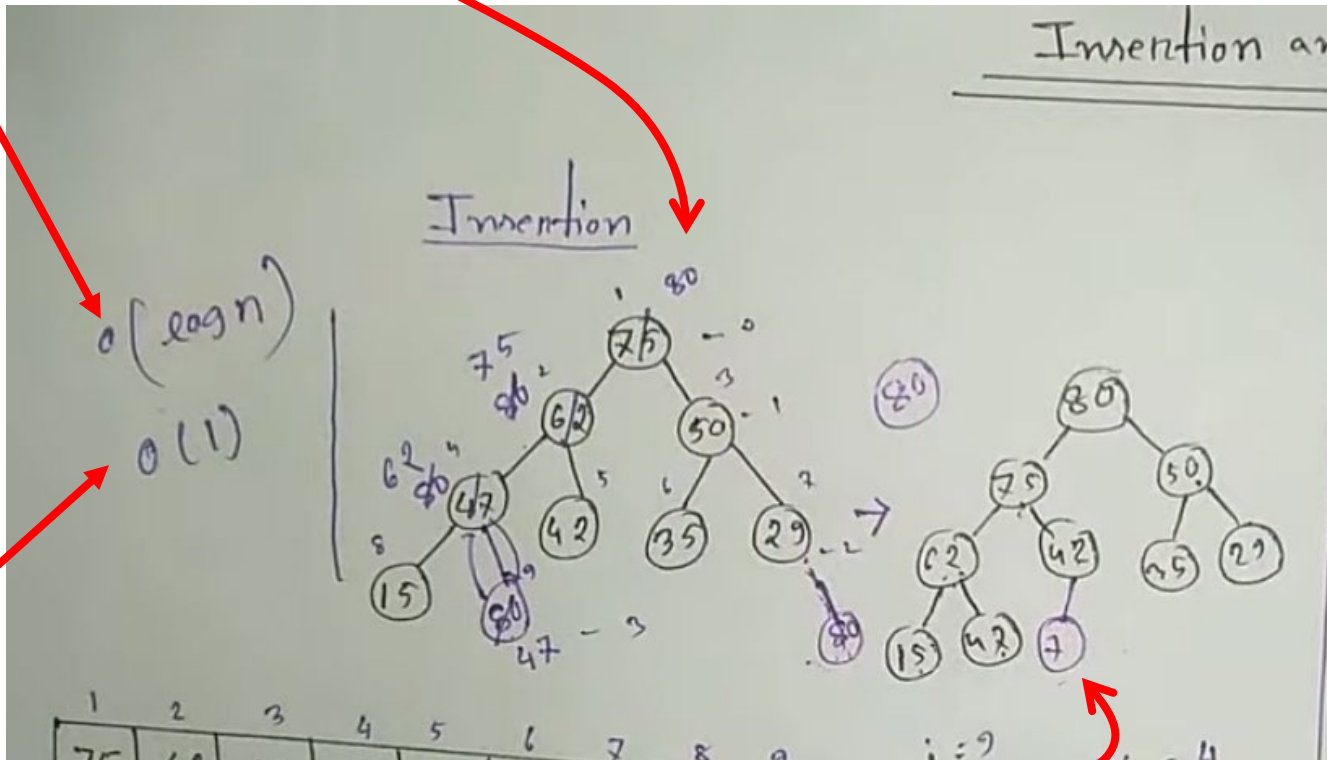
Time complexity of maxheap

Worst case when
"n" no. of swaps
(n=height)

$O(\log n)$

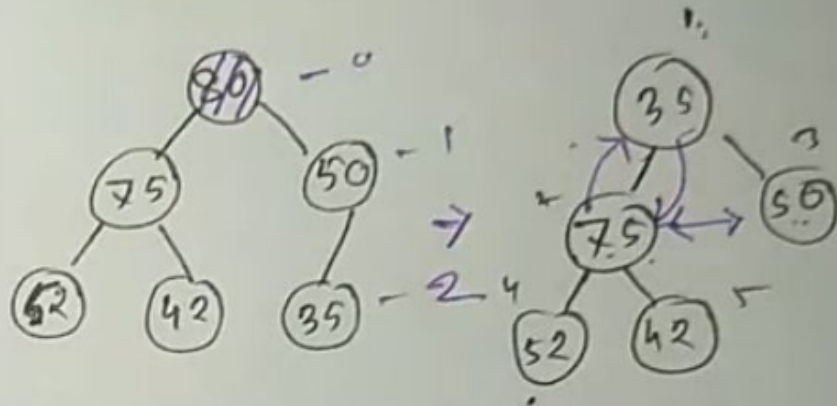
$O(1)$

Best case
when element
inserted at
first try



Deletion

$O(\log n)$

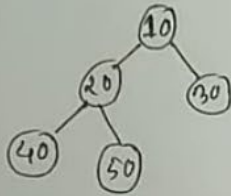


1	2	3	4	5	6
80	75	50	52	42	35

Min Heap

Insertion and Deletion

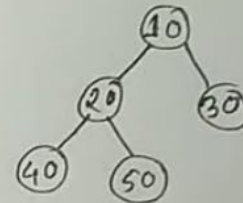
Insertion



1	2	3	4	5	6
10	20	30	40	50	

1	2	3	4	5	6

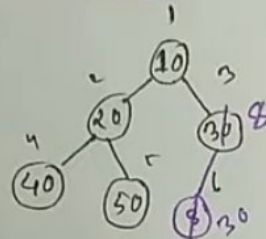
Deletion



1	2	3	4	5
10	20	30	40	50

Min
Invention

Invention



80

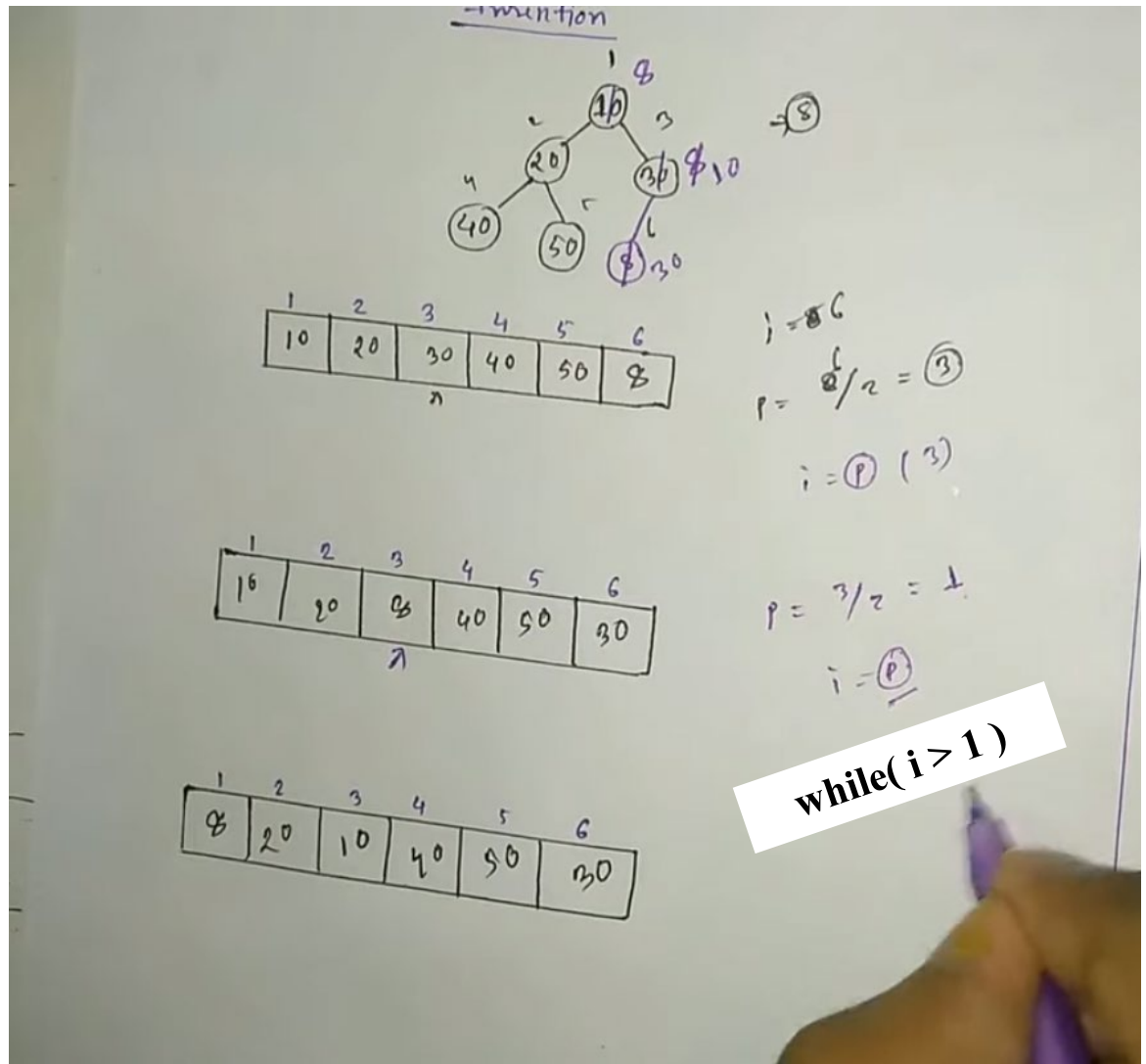
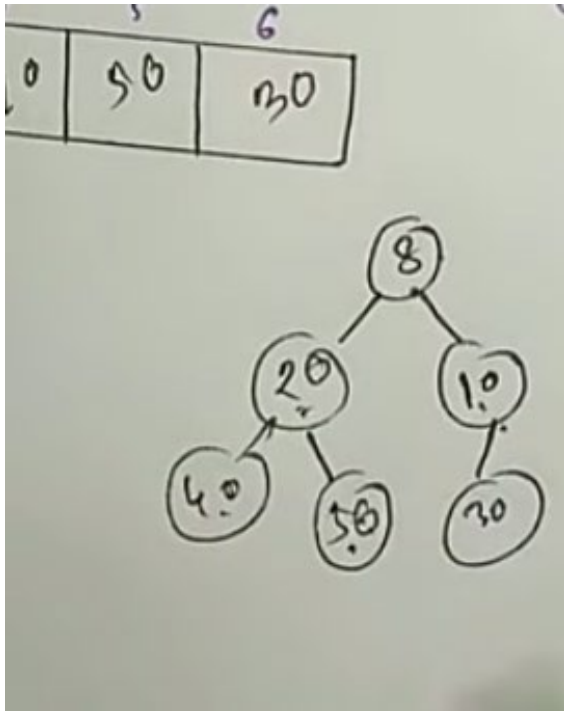
1	2	3	4	5	6
10	20	30	40	50	80

$i = 6$

$$p = \frac{6}{2} = 3$$

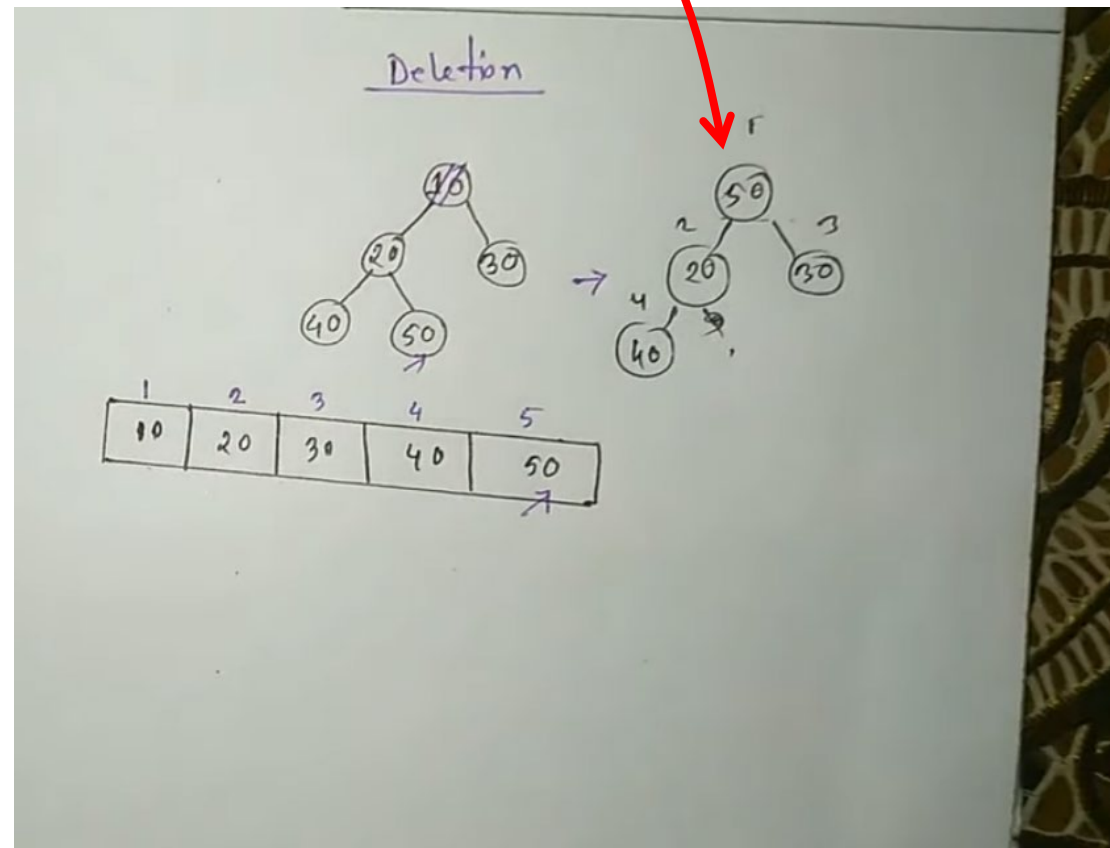
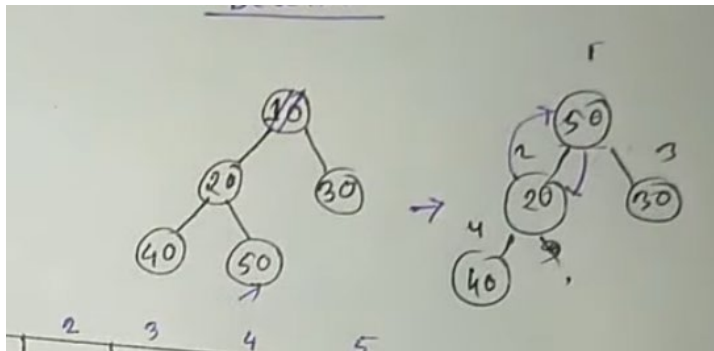
1	2	3	4	5	6
10	20	30	40	50	30

1	2	3	4	5
---	---	---	---	---

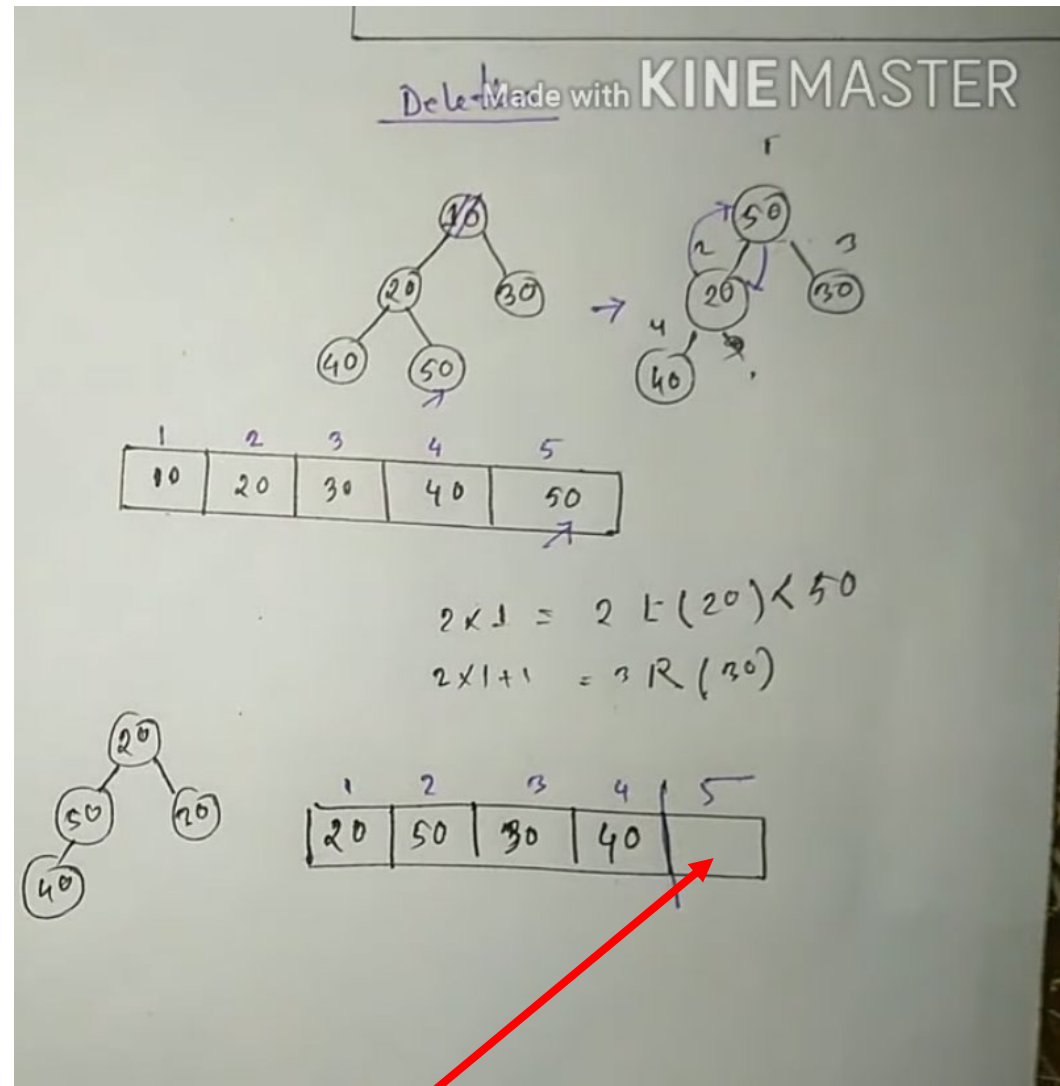


Does not follow
minheap property

Since in minheap parent
cannot be greater than
child, therefore we swap
“the lesser child” with
smaller root in each step



Step 1



Free space for
deleted values

Step 2

Deletion sorts the deleted values in descending order

